

UNIVERSIDAD SERGIO ARBOLEDA

Escuela de Ciencias Exactas e Ingenieria

Documento de formulacion

Fase 1 - Corte 1: Agente Inteligente y Formulacion

Sistema inteligente para apoyo a la planificacion de rutas de emergencia en Bogota

Curso: Inteligencia Artificial - SIST5036

Docente: Joaquin F. Sanchez

Programa: Ciencias de la Computacion e Inteligencia Artificial - Semestre VI

Integrantes:

Santiago Parra Acuña

Sergio Alberto Morales Pirajan

Angel Robles Araque

2026

Tabla de contenido

1. Introduccion
2. Pregunta orientadora
3. Descripcion del problema
4. Objetivos
5. Modelo PEAS del agente
6. Modelo del entorno
7. Dataset inicial y fuentes oficiales
8. Arquitectura del prototipo
9. Funcion de utilidad
10. Pruebas y evidencias
11. Conclusiones parciales y proyeccion
12. Referencias
13. Matriz final de cumplimiento de la Fase 1
14. Anexo A - Evidencia del codigo del grafo
15. Anexo B - Evidencia del prototipo del agente
16. Anexo C - Pruebas ejecutables y salida
17. Anexo D - Presentacion del avance
18. Anexo E - Estructura y ejecucion del repositorio

1. Introduccion

Este documento presenta la formulacion de la Fase 1 del proyecto integrador de Inteligencia Artificial. La solucion propuesta consiste en un sistema inteligente para apoyar la planificacion de atencion de emergencias medicas en Bogota, mediante la representacion del entorno urbano como un grafo y el diseno de un agente basado en utilidad.

A diferencia de un sistema que solo selecciona el hospital mas cercano, el prototipo considera varios criterios: distancia, tiempo estimado, cantidad de semaforos, demanda historica de emergencias por localidad y adecuacion del centro de atencion. El proyecto no utiliza TransMilenio; se centra en red vial, intersecciones y centros de urgencias.

2. Pregunta orientadora

¿Como disenar un agente inteligente capaz de seleccionar un centro de atencion de urgencias y planificar una ruta de emergencia en Bogota, utilizando grafos y datasets oficiales actualizados sobre servicios de salud, llamadas de emergencia, red vial y semaforizacion?

3. Descripcion del problema

En una emergencia medica, la decision de traslado no depende unicamente de la distancia al centro de salud. Tambien pueden influir el tiempo estimado de desplazamiento, la conectividad vial, el numero de intersecciones semaforizadas, la demanda historica de emergencias en la zona y la capacidad del centro para atender urgencias o transporte medicalizado.

Por esta razon, la red se representa como un grafo no dirigido $G = (V, E)$. Los nodos V corresponden a puntos de emergencia, intersecciones viales y centros de atencion. Las aristas E representan conexiones viales disponibles entre nodos. Cada conexion almacena atributos como distancia, tiempo estimado, semaforos y disponibilidad.

4. Objetivos

4.1 Objetivo general

Desarrollar un prototipo inicial de agente inteligente basado en utilidad que represente una red de movilidad de emergencias en Bogota mediante un grafo y seleccione rutas hacia centros de atencion de urgencias usando criterios de eficiencia y adecuacion del servicio.

4.2 Objetivos especificos de Fase 1

- Definir formalmente el problema mediante el modelo PEAS.
- Modelar una red vial simplificada de Bogota mediante un grafo en NetworkX.
- Construir un dataset inicial de nodos y aristas con atributos necesarios para el agente.
- Verificar fuentes oficiales disponibles y actualizadas para servicios de urgencias, Linea 123, red semaforica y malla vial.
- Implementar un prototipo de agente basado en utilidad.
- Validar el prototipo mediante pruebas de seleccion de ruta, comparacion por criterio y bloqueo dinamico.

5. Modelo PEAS del agente

Componente	Descripcion
Performance	Minimizar costo combinado de traslado, reducir distancia y tiempo, evitar demasiadas intersecciones semaforizadas y favorecer centros con mejor adecuacion para urgencias.
Environment	Red vial de Bogota representada mediante grafo: punto de emergencia, intersecciones, hospitales y conexiones disponibles o bloqueadas.
Actuators	Seleccionar el centro de atencion recomendado y avanzar por los tramos de la ruta calculada hasta llegar al destino.
Sensors	Percibir nodo actual, vecinos disponibles, atributos de cada tramo, estado de disponibilidad, demanda historica por localidad y características del centro de salud.

6. Modelo del entorno

El entorno se implementa como un grafo no dirigido $G = (V, E)$. En el dataset inicial hay 13 nodos y 16 conexiones. Los nodos tienen tipo, localidad, latitud, longitud y descripcion. Las aristas tienen distancia, tiempo estimado, numero de semaforos, disponibilidad y tipo de via.

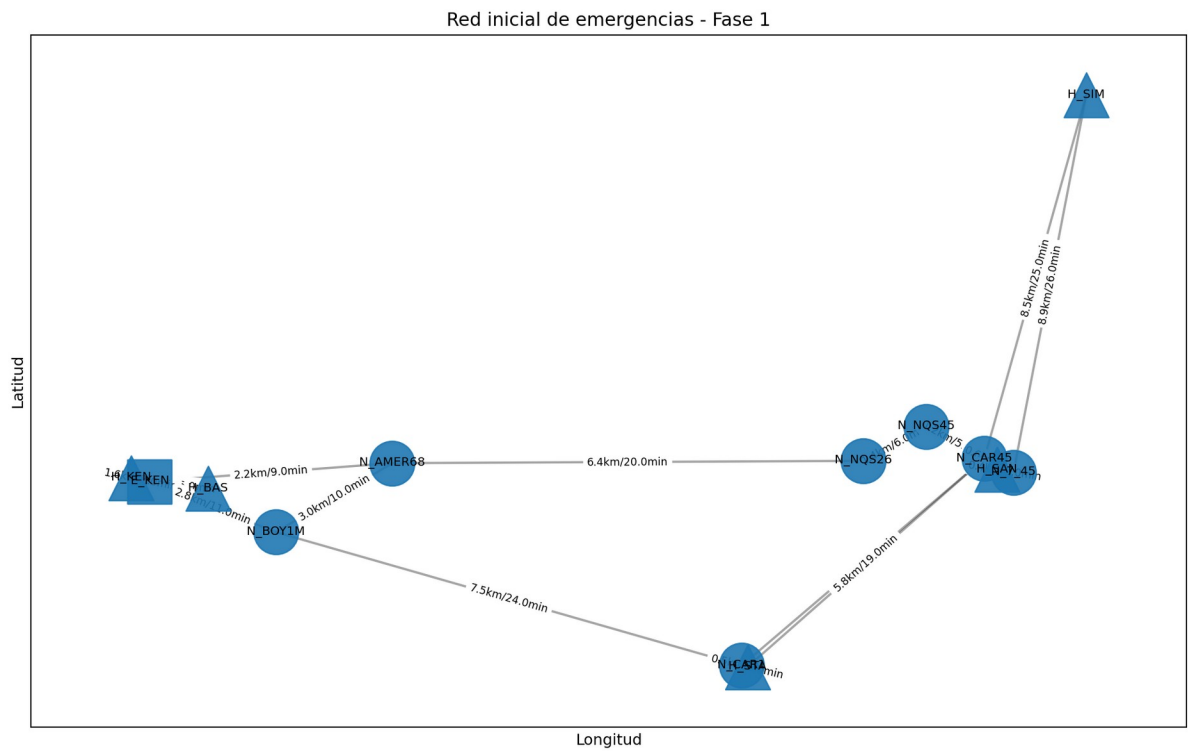


Figura 1. Red inicial de emergencias usada en el prototipo de Fase 1.

7. Dataset inicial y fuentes oficiales

El prototipo incluye un dataset inicial reducido para ejecutar pruebas locales. Este dataset fue normalizado en archivos CSV y esta acompañado por la verificación de fuentes oficiales actualizadas. En una fase posterior puede reemplazarse por los CSV/GeoJSON completos descargados desde los portales oficiales.

Dataset	Uso en el proyecto	Actualizacion verificada
Servicios de Urgencias y Ambulancias - SDS	Validar centros con urgencias, transporte basico y transporte	23 de julio de 2026

	medicalizado.	
Linea 123 Salud - SDS	Analizar demanda historica de emergencias por localidad.	4 de septiembre de 2026
Red Semaforica - SDM	Agregar atributo de intersecciones y semaforos al entorno.	Dato 12 de junio de 2026; metadato 2 de septiembre de 2026
Malla Vial Integral - IDECA/Movilidad	Base oficial para escalar el grafo vial completo.	Fuente oficial vigente consultada el 9 de septiembre de 2026

7.1 Dataset inicial - nodos

Campo	Descripcion
id	Identificador corto del nodo
nombre	Nombre del punto, interseccion u hospital
tipo	emergencia, interseccion u hospital
localidad	Localidad de Bogota
lat/lon	Coordenadas aproximadas para visualizacion
descripcion	Contexto del nodo

7.2 Dataset inicial - aristas

Campo	Descripcion
origen/destino	Nodos conectados
distancia_km	Distancia estimada del tramo
tiempo_min	Tiempo estimado de desplazamiento
semaforos	Cantidad aproximada de intersecciones semaforizadas
disponible	Permite simular bloqueo dinamico
tipo_via	arterial, vial local o acceso

8. Arquitectura del prototipo

La solucion se implemento en Python con NetworkX y se organizo en modulos para separar responsabilidades:

- grafo.py: carga datasets CSV, construye el grafo y expone vecinos disponibles, bloqueo de vias y hospitales disponibles.
- agente.py: implementa el agente basado en utilidad, evalua rutas candidatas y selecciona la mejor alternativa.
- pruebas.py: ejecuta escenarios de validacion y genera salida_pruebas.txt.
- visualizar.py: genera las figuras del grafo y de las rutas usando Matplotlib.
- main.py: punto de entrada para ejecutar el prototipo.

9. Funcion de utilidad

Para cada tramo, el agente calcula un costo ponderado. Un menor valor representa una ruta mas conveniente dentro del modelo inicial:

$$\text{costo} = 0.35 * \text{distancia} + 0.30 * (\text{tiempo} / 10) + 0.15 * (\text{semaforos} / 5) + 0.10 * \text{demanda} - 0.25 * \text{adecuacion}$$

Criterio	Peso	Justificacion
Distancia	0.35	Mantiene la eficiencia del recorrido.
Tiempo	0.30	Aproxima la urgencia del desplazamiento.
Semaforos	0.15	Penaliza tramos con mas detenciones potenciales.
Demanda historica	0.10	Representa presion historica de emergencias por localidad.
Adecuacion del centro	0.25	Bonifica centros con servicios mas apropiados para urgencias.

10. Pruebas y evidencias

Prueba	Descripcion	Resultado
1	Ruta normal desde emergencia simulada en Kennedy.	El agente selecciona Hospital Occidente de Kennedy.
2	Comparacion entre seleccion por distancia y seleccion por utilidad.	El punto mas cercano es basico, pero la utilidad favorece el hospital integral.
3	Bloqueo dinamico de accesos cercanos.	El agente recalcula y selecciona una ruta alternativa hacia Hospital Santa Clara.

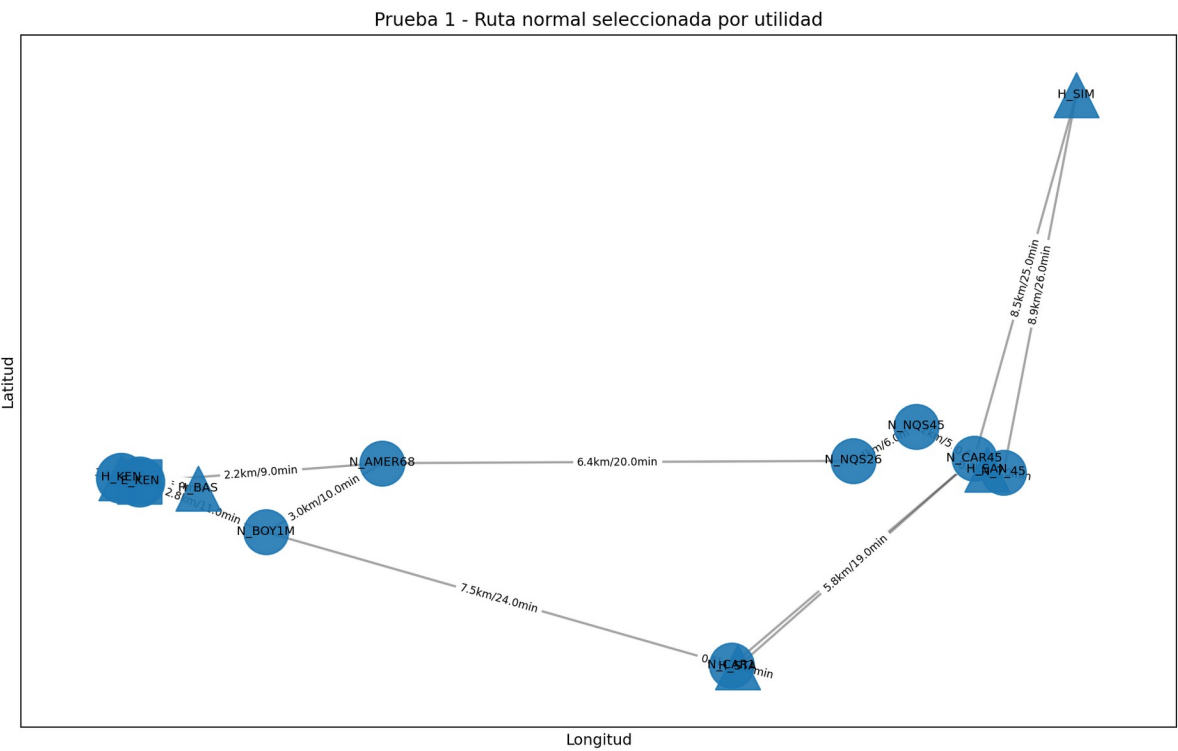


Figura 2. Ruta normal seleccionada por el agente.

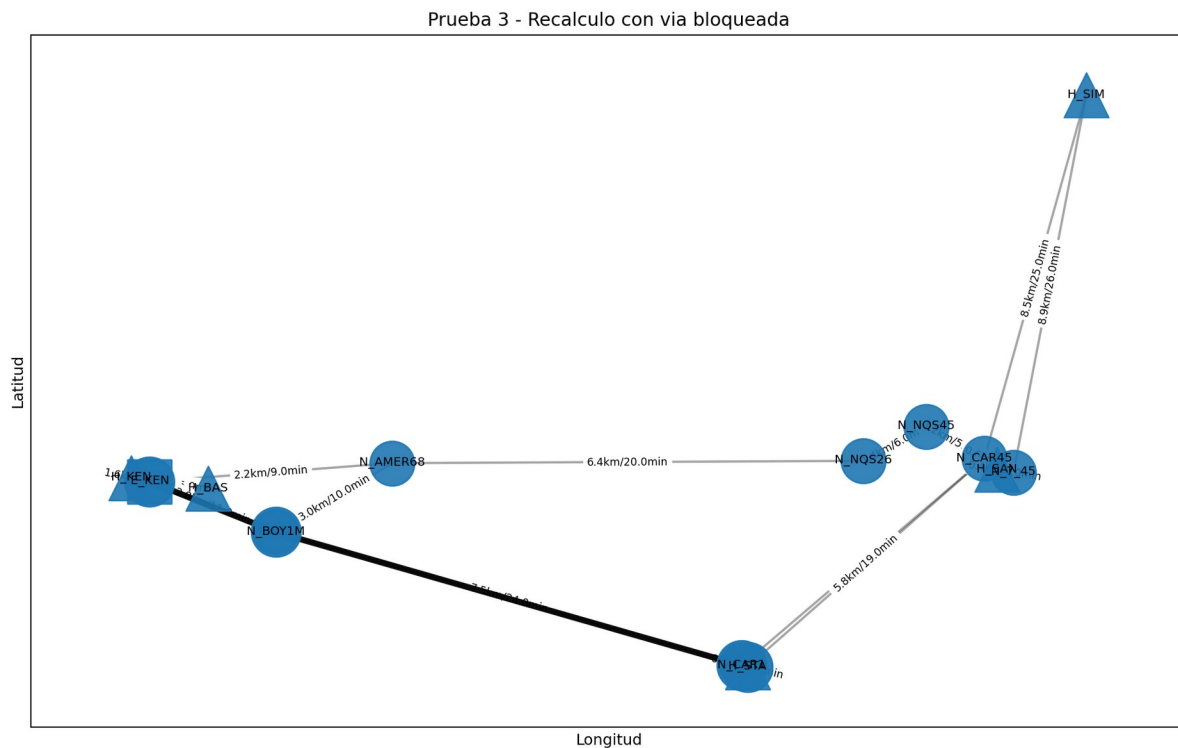


Figura 3. Recalculo de ruta ante bloqueo dinámico.

11. Conclusiones parciales y proyeccion

La Fase 1 permite demostrar que el problema puede formularse como una red de movilidad representada mediante grafos. El agente percibe el estado del entorno, evalúa alternativas y selecciona una ruta con base en una función de utilidad, no solamente por distancia.

Para la Fase 2, el mismo grafo servirá para implementar y comparar BFS, DFS, UCS, búsqueda voraz y A*. Para la Fase 3, los datos de Línea 123 y servicios de salud permiten plantear tareas de clasificación, regresión o agrupamiento, sin depender de TransMilenio ni de datos inventados.

12. Referencias

- Secretaría Distrital de Salud. (2026). Servicios de Urgencias y Ambulancias. Datos Abiertos Bogotá.
- Secretaría Distrital de Salud. (2026). Llamadas de Urgencias y Emergencias que ingresan a través de la Línea 123. Datos Abiertos Bogotá.
- Secretaría Distrital de Movilidad. (2026). Red Semafórica de Bogotá D.C. Datos Abiertos Bogotá.
- IDECA. (2026). Malla Vial Integral Bogotá D.C. Datos Abiertos / Mapas Bogotá.
- Hagberg, A., Schult, D., & Swart, P. (2008). Exploring network structure, dynamics, and function using NetworkX. Proceedings of the 7th Python in Science Conference.
- Russell, S., & Norvig, P. (2020). Artificial Intelligence: A Modern Approach (4th ed.). Pearson.
- Universidad Sergio Arboleda. (2026). Presentación del proyecto de la asignatura Inteligencia Artificial - SIST5036.

13. Matriz final de cumplimiento de la Fase 1

Esta matriz consolida los entregables exigidos para el Corte 1 y muestra la evidencia incluida tanto dentro de este documento como en el repositorio entregable. Los componentes que antes requerian verificacion (codigo, prototipo, pruebas y presentacion) quedan documentados en los anexos siguientes.

Lo exigido en Fase 1	Estado	Observacion / evidencia
Documento de formulacion	CUMPLE	El documento contiene portada, introduccion, formulacion, objetivos, PEAS, entorno, datasets, arquitectura, pruebas, conclusiones y referencias.
Definicion/descrip. del problema	CUMPLE	Se formula la planificacion de rutas de emergencia en Bogota sin depender de TransMilenio.
Pregunta orientadora	CUMPLE	Pregunta definida especificamente para seleccion de centro de urgencias y planificacion de ruta con grafos y datos oficiales.
Objetivos	CUMPLE	Incluye objetivo general y objetivos especificos exclusivos para la Fase 1.
Modelo PEAS	CUMPLE	Performance, Environment, Actuators y Sensors estan definidos y vinculados al comportamiento del agente.
Representacion mediante grafo	CUMPLE	Se define $G=(V,E)$ y se implementa en NetworkX con nodos de emergencia, intersecciones y hospitales.
Diagrama de la red	CUMPLE	La Figura 1 muestra el grafo inicial; las Figuras 2 y 3 muestran rutas de prueba y recalcu.
Dataset inicial	CUMPLE	Se incluyen data/nodos.csv, data/aristas.csv, muestra de servicios de urgencias y resumen de Linea 123, junto con la trazabilidad de fuentes.
Codigo del grafo	CUMPLE	Existe src/grafos.py. El Anexo A incorpora evidencia del codigo que carga CSV, construye el grafo y administra bloqueos.
Prototipo del agente	CUMPLE	Existe src/agente.py. El Anexo B documenta la clase AgenteEmergenciasUtilidad y su seleccion por utilidad.
Pruebas	CUMPLE	src/pruebas.py ejecuta tres escenarios. La ejecucion fue validada y genera output/salida_pruebas.txt.
Presentacion del avance	CUMPLE	Se incluyen slides/Presentacion_Fase1_Emergencias_Bogota.pptx y su version PDF. El Anexo D deja evidencia de su contenido.

Nota de trazabilidad de datos: el repositorio de Fase 1 trabaja con un dataset inicial reducido y normalizado para validar la arquitectura y el comportamiento del agente. Las fuentes oficiales verificadas se documentan en data/fuentes_datos_verificadas.md. Los valores del prototipo no deben interpretarse como informacion operacional en tiempo real; la ampliacion con los recursos oficiales completos queda preparada para las siguientes fases.

14. Anexo A - Evidencia del código del grafo

El archivo `src/graf.py` implementa la representación $G=(V,E)$ mediante NetworkX. Carga los CSV de nodos, aristas, servicios y demanda; agrega atributos a nodos y conexiones; permite consultar hospitales y vecinos; y soporta el bloqueo/habilitación dinámica de vías.

Código completo de `src/graf.py`

```
from __future__ import annotations

from pathlib import Path
from typing import Dict, Iterable, List, Tuple

import pandas as pd
import networkx as nx

class EntornoEmergencias:
    """Entorno de movilidad para atención de emergencias.

    Construye un grafo no dirigido  $G=(V,E)$ . Los nodos representan puntos de
    emergencia, intersecciones y hospitales. Las aristas representan tramos
    viales con atributos de distancia, tiempo, semáforos y disponibilidad.
    """

    def __init__(self, data_dir: str | Path):
        self.data_dir = Path(data_dir)
        self.nodos_df = pd.read_csv(self.data_dir / "nodos.csv")
        self.aristas_df = pd.read_csv(self.data_dir / "aristas.csv")
        self.servicios_df = pd.read_csv(self.data_dir / "servicios_urgencias_muestra.csv")
        self.demanda_df = pd.read_csv(self.data_dir / "llamadas123_resumen_localidad_muestra.csv")
        self.grafo = nx.Graph()
        self._construir_grafo()

    def _construir_grafo(self) -> None:
        demanda = {
            row["localidad"]: float(row["indice_demanda_0_1"])
            for _, row in self.demanda_df.iterrows()
        }
        servicios = self.servicios_df.set_index("id_hospital").to_dict("index")

        for _, row in self.nodos_df.iterrows():
            node_id = row["id"]
            attrs = row.to_dict()
            attrs["demanda_localidad"] = demanda.get(row["localidad"], 0.30)
            if node_id in servicios:
                attrs.update(servicios[node_id])
            self.grafo.add_node(node_id, **attrs)

        for _, row in self.aristas_df.iterrows():
            disponible = str(row["disponible"]).strip().lower() in {"si", "sí", "true", "1"}
            self.grafo.add_edge(
                row["origen"],
                row["destino"],
                distancia_km=float(row["distancia_km"]),
                tiempo_min=float(row["tiempo_min"]),
                semaforos=int(row["semaforos"]),
                disponible=disponible,
                tipo_via=row["tipo_via"],
                observacion=row["observacion"],
            )

    def hospitales_disponibles(self) -> List[str]:
        return [n for n, data in self.grafo.nodes(data=True) if data.get("tipo") == "hospital"]

    def vecinos_disponibles(self, nodo: str) -> List[str]:
        return [
            vecino
            for vecino in self.grafo.neighbors(nodo)
            if self.grafo[nodo][vecino].get("disponible", True)
        ]
```

```
def bloquear_via(self, origen: str, destino: str) -> None:
    if self.grafo.has_edge(origen, destino):
        self.grafo[origen][destino]["disponible"] = False

def habilitar_via(self, origen: str, destino: str) -> None:
    if self.grafo.has_edge(origen, destino):
        self.grafo[origen][destino]["disponible"] = True

def subgrafo_disponible(self) -> nx.Graph:
    g = nx.Graph()
    for n, attrs in self.grafo.nodes(data=True):
        g.add_node(n, **attrs)
    for u, v, attrs in self.grafo.edges(data=True):
        if attrs.get("disponible", True):
            g.add_edge(u, v, **attrs)
    return g

def resumen(self) -> Dict[str, int]:
    return {
        "nodos": self.grafo.number_of_nodes(),
        "aristas": self.grafo.number_of_edges(),
        "hospitales": len(self.hospitales_disponibles()),
    }
```

15. Anexo B - Evidencia del prototipo del agente

El archivo src/agente.py contiene el prototipo inicial del agente basado en utilidad. La clase AgenteEmergenciasUtilidad percibe el subgrafo disponible, calcula el costo de las conexiones, evalúa rutas hacia cada hospital candidato y selecciona la alternativa con menor utilidad ajustada.

Código completo de src/agente.py

```
from __future__ import annotations

from dataclasses import dataclass
from typing import Dict, List, Tuple

import networkx as nx

from grafo import EntornoEmergencias

@dataclass
class ResultadoRuta:
    hospital: str
    ruta: List[str]
    costo_total: float
    distancia_total: float
    tiempo_total: float
    semaforos_total: int
    utilidad: float

class AgenteEmergenciasUtilidad:
    """Agente basado en utilidad para seleccionar centro de urgencias y ruta.

    El agente percibe el grafo disponible, evalúa rutas posibles hacia centros
    de urgencias y elige la alternativa con menor costo ajustado. En Fase 1 se
    usa Dijkstra ponderado como mecanismo inicial de decisión; en Fase 2 se
    reemplazara/comparara con BFS, DFS, UCS, Greedy y A*.
    """

    def __init__(self, entorno: EntornoEmergencias, pesos: Dict[str, float] | None = None):
        self.entorno = entorno
        self.pesos = pesos or {
            "distancia": 0.35,
            "tiempo": 0.30,
            "semaforos": 0.15,
            "demanda": 0.10,
            "adecuacion": 0.25,
        }

    def costo_arista(self, u: str, v: str, attrs: Dict) -> float:
        localidad_destino = self.entorno.grafo.nodes[v].get("localidad", "")
        demanda = float(self.entorno.grafo.nodes[v].get("demanda_localidad", 0.30))
        return (
            self.pesos["distancia"] * float(attrs["distancia_km"])
            + self.pesos["tiempo"] * (float(attrs["tiempo_min"]) / 10)
            + self.pesos["semaforos"] * (int(attrs["semaforos"]) / 5)
            + self.pesos["demanda"] * demanda
        )

    def evaluar_ruta(self, origen: str, hospital: str) -> ResultadoRuta | None:
        g = self.entorno.subgrafo_disponible()
        try:
            ruta = nx.shortest_path(g, origen, hospital, weight=self.costo_arista)
        except (nx.NetworkXNoPath, nx.NodeNotFound):
            return None

        distancia = 0.0
        tiempo = 0.0
        semaforos = 0
        costo = 0.0
        for u, v in zip(ruta[:-1], ruta[1:]):
            attrs = g[u][v]
            distancia += float(attrs["distancia_km"])
```

```

        tiempo += float(attrs["tiempo_min"])
        semaforos += int(attrs["semaforos"])
        costo += self.costo_arista(u, v, attrs)

    datos_hospital = g.nodes[hospital]
    adecuacion = float(datos_hospital.get("nivel_adecuacion", 3))
    # Mientras mayor sea la adecuacion del centro, menor queda el costo ajustado.
    utilidad = costo - self.pesos["adecuacion"] * adecuacion

    return ResultadoRuta(
        hospital=hospital,
        ruta=ruta,
        costo_total=round(costo, 4),
        distancia_total=round(distancia, 2),
        tiempo_total=round(tiempo, 2),
        semaforos_total=semaforos,
        utilidad=round(utilidad, 4),
    )

def seleccionar_mejor_ruta(self, origen: str) -> ResultadoRuta:
    candidatos = []
    for hospital in self.entorno.hospitales_disponibles():
        resultado = self.evaluar_ruta(origen, hospital)
        if resultado is not None:
            candidatos.append(resultado)
    if not candidatos:
        raise ValueError("No existe ruta disponible hacia ningun hospital.")
    return min(candidatos, key=lambda r: r.utilidad)

def explicar_decision(self, resultado: ResultadoRuta) -> str:
    nombres = [self.entorno.grafo.nodes[n]["nombre"] for n in resultado.ruta]
    return (
        f"Hospital seleccionado: {self.entorno.grafo.nodes[resultado.hospital]['nombre']}\n"
        f"Ruta: {' -> '.join(resultado.ruta)}\n"
        f"Ruta detallada: {' -> '.join(nombres)}\n"
        f"Distancia total: {resultado.distancia_total} km\n"
        f"Tiempo estimado: {resultado.tiempo_total} min\n"
        f"Semaforos estimados: {resultado.semaforos_total}\n"
        f"Costo combinado: {resultado.costo_total}\n"
        f"Utilidad ajustada: {resultado.utilidad}\n"
    )

```

16. Anexo C - Pruebas ejecutables y salida

Las pruebas no son solamente descritas en el informe: se encuentran automatizadas en `src/pruebas.py`. El script valida el resumen del entorno, una ruta normal, una comparacion entre cercania y utilidad y el recalcule ante un bloqueo dinamico.

Codigo completo de `src/pruebas.py`

```
from __future__ import annotations

from pathlib import Path

from agente import AgenteEmergenciasUtilidad
from grafo import EntornoEmergencias

def ejecutar_pruebas(data_dir: str | Path, output_file: str | Path | None = None) -> str:
    entorno = EntornoEmergencias(data_dir)
    agente = AgenteEmergenciasUtilidad(entorno)
    lineas = []

    lineas.append("===== RESUMEN DEL ENTORNO =====")
    lineas.append(str(entorno.resumen()))
    lineas.append("")

    lineas.append("===== PRUEBA 1: Ruta normal =====")
    r1 = agente.seleccionar_mejor_ruta("E_KEN")
    lineas.append(agente.explicar_decision(r1))

    lineas.append("===== PRUEBA 2: Comparacion por criterio =====")
    # Criterio simple: hospital con menor distancia de ruta sin bonificar adecuacion.
    resultados = [agente.evaluar_ruta("E_KEN", h) for h in entorno.hospitales_disponibles()]
    resultados = [r for r in resultados if r is not None]
    por_distancia = min(resultados, key=lambda r: r.distancia_total)
    por_utilidad = min(resultados, key=lambda r: r.utilidad)
    lineas.append(f"Centro mas cercano por distancia: {por_distancia.hospital} ({por_distancia.distancia_total} km)")
    lineas.append(f"Centro elegido por utilidad: {por_utilidad.hospital} (utilidad {por_utilidad.utilidad})")
    lineas.append("Interpretacion: el agente puede considerar tiempo, semaforos, demanda local y adecuacion del servicio, no solo distancia.")
    lineas.append("")

    lineas.append("===== PRUEBA 3: Bloqueo dinamico =====")
    entorno.bloquear_via("E_KEN", "H_KEN")
    entorno.bloquear_via("E_KEN", "H_BAS")
    r3 = agente.seleccionar_mejor_ruta("E_KEN")
    lineas.append("Se bloquearon los accesos E_KEN - H_KEN y E_KEN - H_BAS para simular afectacion en la zona cercana.")
    lineas.append(agente.explicar_decision(r3))
    lineas.append("Interpretacion: el agente percibe que la conexion ya no esta disponible y recalcula una ruta valida.")

    salida = "\n".join(lineas)
    if output_file:
        Path(output_file).parent.mkdir(parents=True, exist_ok=True)
        Path(output_file).write_text(salida, encoding="utf-8")
    return salida

if __name__ == "__main__":
    repo = Path(__file__).resolve().parents[1]
    print(ejecutar_pruebas(repo / "data", repo / "output" / "salida_pruebas.txt"))
```

16.1 Salida verificada de las pruebas

Resultado generado en `output/salida_pruebas.txt`

```
===== RESUMEN DEL ENTORNO =====
{'nodos': 13, 'aristas': 16, 'hospitales': 5}
```

```
===== PRUEBA 1: Ruta normal =====
Hospital seleccionado: Hospital Occidente de Kennedy
Ruta: E_KEN -> H_KEN
Ruta detallada: Emergencia simulada - Kennedy -> Hospital Occidente de Kennedy
Distancia total: 1.6 km
Tiempo estimado: 7.0 min
Semaforos estimados: 4
Costo combinado: 0.975
Utilidad ajustada: -0.275

===== PRUEBA 2: Comparacion por criterio =====
Centro mas cercano por distancia: H_BAS (0.7 km)
Centro elegido por utilidad: H_KEN (utilidad -0.275)
Interpretacion: el agente puede considerar tiempo, semaforos, demanda local y adecuacion del servicio, no solo
distancia.

===== PRUEBA 3: Bloqueo dinamico =====
Se bloquearon los accesos E_KEN - H_KEN y E_KEN - H_BAS para simular afectacion en la zona cercana.
Hospital seleccionado: Hospital Santa Clara
Ruta: E_KEN -> N_BOY1M -> N_CAR1 -> H_STA
Ruta detallada: Emergencia simulada - Kennedy -> Av Boyaca x Primero de Mayo -> Av Caracas x Calle 1 -> Hospital
Santa Clara
Distancia total: 10.8 km
Tiempo estimado: 38.0 min
Semaforos estimados: 21
Costo combinado: 5.715
Utilidad ajustada: 4.465

Interpretacion: el agente percibe que la conexion ya no esta disponible y recalcula una ruta valida.
```

Resultado de validacion: las tres pruebas finalizaron correctamente. El entorno reporto 13 nodos, 16 aristas y 5 nodos de tipo hospital. En la prueba de bloqueo, el agente recalculo una ruta alternativa hacia Hospital Santa Clara sin utilizar las dos conexiones bloqueadas.

17. Anexo D - Presentacion del avance

La presentacion del avance se entrega en dos formatos:

slides/Presentacion_Fase1_Emergencias_Bogota.pptx y

slides/Presentacion_Fase1_Emergencias_Bogota.pdf. Contiene 10 diapositivas que resumen la Fase 1 y permiten sustentar el proyecto.

Fase 1 - Agente inteligente y formulacion

Sistema inteligente para apoyo a la planificacion de rutas de emergencia en Bogota

Curso: Inteligencia Artificial - SIST5036 | Corte 1

Integrantes: Santiago Parra Acuña | Sergio Alberto Morales Pirajan | Angel Robles Araque

Sin TransMilenio: red vial, centros de urgencias y datos oficiales

Figura 4. Portada de la presentacion del avance incluida en el repositorio.

- 1. Portada e identificacion del proyecto e integrantes.
- 2. Problema y pregunta orientadora.
- 3. Fuentes oficiales verificadas.
- 4. Modelo PEAS del agente.
- 5. Modelo del entorno y grafo $G=(V,E)$.
- 6. Funcion de utilidad.
- 7. Prueba 1: ruta normal.
- 8. Pruebas 2 y 3: comparacion y bloqueo dinamico.
- 9. Arquitectura del prototipo.
- 10. Conclusiones y proyeccion a las Fases 2 y 3.

18. Anexo E - Estructura y ejecucion del repositorio

La organizacion de archivos separa datos, codigo, figuras, resultados, documentacion y presentacion. Esta estructura facilita la reproducibilidad y la evaluacion del proyecto.

Estructura de entrega

```
Fase1_Emergencias_Bogota/
├── README.md
├── requirements.txt
├── data/
│   ├── nodos.csv
│   ├── aristas.csv
│   ├── servicios_urgencias_muestra.csv
│   ├── llamadas123_resumen_localidad_muestra.csv
│   └── fuentes_datos_verificadas.md
├── src/
│   ├── grafo.py
│   ├── agente.py
│   ├── pruebas.py
│   ├── visualizar.py
│   └── main.py
├── figures/
│   ├── grafo_red_emergencias.png
│   ├── ruta_normal.png
│   └── ruta_bloqueo.png
├── output/
│   └── salida_pruebas.txt
├── docs/
│   ├── Fase1_Emergencias_Bogota.docx
│   └── Fase1_Emergencias_Bogota.pdf
└── slides/
    ├── Presentacion_Fase1_Emergencias_Bogota.pptx
    └── Presentacion_Fase1_Emergencias_Bogota.pdf
```

18.1 Ejecucion

Desde la raiz del proyecto, instalar las dependencias y ejecutar el prototipo con los siguientes comandos:

```
pip install -r requirements.txt
python src/main.py
python src/visualizar.py
```

El primer comando instala pandas, NetworkX y Matplotlib segun requirements.txt. main.py ejecuta las pruebas y genera la salida textual; visualizar.py genera las figuras del grafo y de las rutas utilizadas como evidencia.

18.2 Cierre de Fase 1

Con estas evidencias, la entrega contiene los componentes solicitados para el Corte 1: documento de formulacion, modelo PEAS, representacion y diagrama de la red, dataset inicial, codigo del grafo, prototipo del agente, pruebas y presentacion del avance. Los algoritmos BFS, DFS, UCS, busqueda voraz y A*, asi como los modelos de aprendizaje automatico, se mantienen fuera de esta entrega y corresponden a las fases posteriores.